1. Título del proyecto + integrantes del equipo

Nombre del proyecto

Skin-disease-detection-ml

Integrantes y rol principal

- Álvaro Guirado Cárdenas → Desarrollo front-end, DevOps
- Diego Hernando Torralba → Desarrollo back-end, API
- Javier García Hernández → ML, Data Engineering, Monitorización

2. Resumen ejecutivo

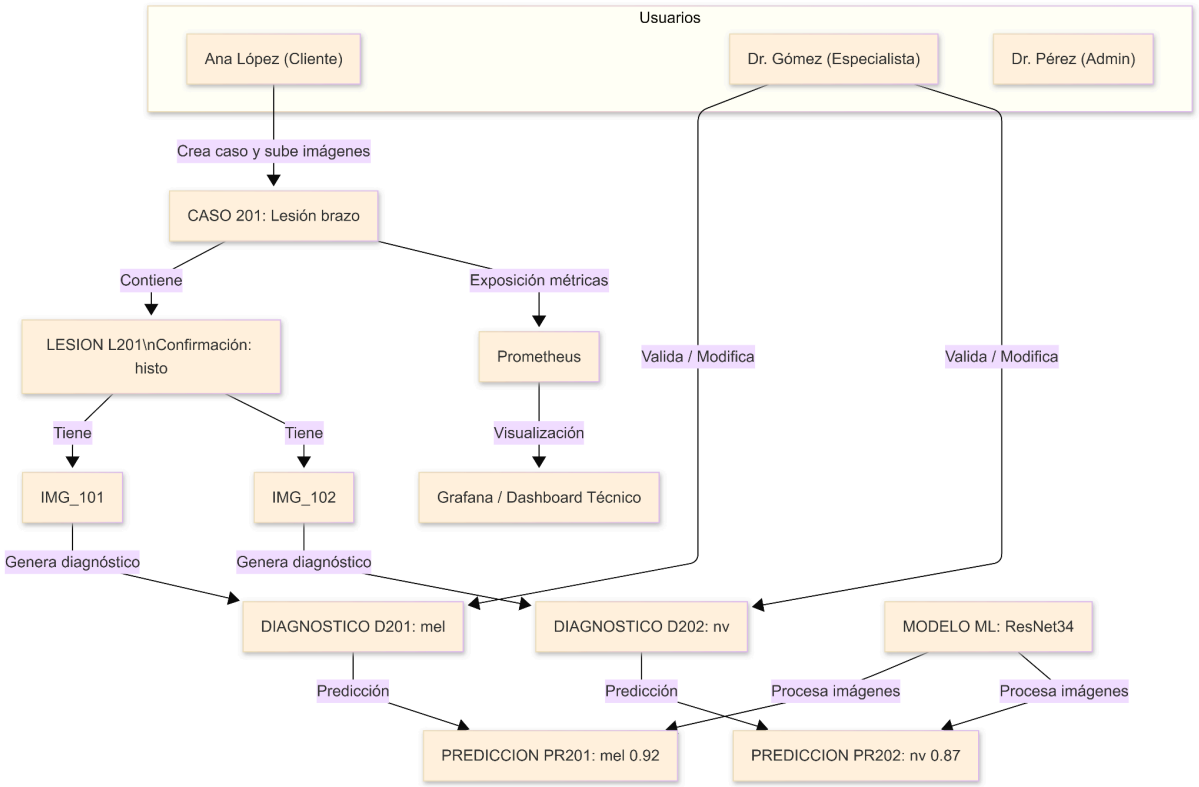
Las enfermedades cutáneas, en especial el cáncer de piel, son de las más comunes a nivel mundial, además con una detección temprana la tasa de supervivencia aumenta a gran escala. Por otro lado, la falta de personal hace que las listas de espera para dermatología sean muy largas, esto, sumado a posibles errores humanos debido a la sobrecarga y cansancio del personal sanitario hace que un simple diagnóstico se convierta en una tarea crítica. Por ello hemos desarrollado un modelo IA capaz de analizar imágenes de piel humana con la finalidad de apoyar a los dermatólogos facilitando un diagnóstico rápido y preciso, permitiendo acelerar resultados, además de ordenar y clasificar diagnósticos automáticamente en base a la rapidez con la que los pacientes deben ser tratados.

3. Descripción funcional del sistema

La aplicación consta de un front en web mediante la cual los usuarios son capaces de subir una foto. Esa foto la recibirá un back creado en Java+Spring Boot (se formateará la imagen en caso de ser necesario). Para mandarla posteriormente al modelo, que realizará el análisis y dará un diagnóstico. El diagnóstico volverá al back para mostrarse en pantalla.

Flujo general de uso

- Ingreso y almacenamiento de los datos: El usuario entra al sistema con sus credenciales (rol cliente), sube una o varias imágenes y se genera un caso nuevo con esas imágenes y datos relevantes (sexo, edad)
- Comunicación con el modelo: Las imágenes se procesan en un back de java + Spring Boot que modifica las características para que las entienda el modelo.
- Integración del modelo: El modelo recibe las imágenes y realiza un diagnóstico que posteriormente se mostrará al usuario en la web.
- Captura de métricas: Prometheus extrae métricas de rendimiento técnico, que posteriormente alimentarán paneles de monitoreo en Grafana.
- Validación médica: Un especialista juega el diagnóstico dado por el modelo y lo modifica en caso de ser erróneo o lo valida si es correcto.



Casos principales de usuario

Caso de uso	Subir Imagen para diagnóstico
Actores	Cliente
Precondiciones	El usuario debe estar registrado en el sistema
Postcondiciones	El usuario recibe un diagnóstico
Propósito	Recibir un diagnóstico
Resúmen	El usuario sube una/varias imágen/es que posteriormente analiza el modelo para dar un diagnóstico

Caso de uso	Acceder al historial de casos
Actores	Cliente
Precondiciones	El usuario debe estar registrado en el sistema
Postcondiciones	El usuario ha accedido al sistema a través de sus credenciales
Propósito	Visualizar los casos médicos
Resúmen	El usuario puede sus casos médicos (los que se crean cuando el usuario sube imágenes)

Caso de uso	Acceder al historial de casos
Actores	Médico
Precondiciones	El usuario debe estar registrado en el sistema
Postcondiciones	El usuario es capaz de visualizar el historial de casos
Propósito	Visualizar todos los casos médicos
Resúmen	El usuario puede ver todos los casos médicos (se pueden ordenar por fecha o por importancia)

Caso de uso	Realizar o modificar un diagnóstico
Actores	Médico
Precondiciones	El usuario debe estar registrado en el sistema y visualizando un caso médico en concreto
Postcondiciones	Se asigna el nuevo diagnóstico al caso
Propósito	Corregir o confirmar diagnósticos realizados por el modelo
Resúmen	El usuario puede modificar o realizar un diagnóstico de un caso en concreto, si el nuevo diagnóstico es diferente al dado por el modelo siempre se asignará el diagnóstico realizado por el médico

Caso de uso	Acceder a métricas de uso
Actores	Admin
Precondiciones	El usuario debe estar registrado en el sistema
Postcondiciones	El usuario es capaz de ver los paneles grafana con el rendimiento técnico
Propósito	Visualizar las métricas de flujo de la aplicación
Resúmen	El usuario sube una/varias imágen/es que posteriormente analiza el modelo para dar un diagnóstico

<u>El sistema puede:</u>

- Registrar nuevos usuarios con el rol “Cliente”
- Para Cliente:
    - Subir una o varias imágenes para crear un nuevo caso.
    - Mostrar el historial de todos los casos en los que ha participado.
    - Seleccionar un caso para ver el diagnóstico y la información relacionada.

- Para Médico:
    - Ver todos los casos con los datos de los pacientes/clientes ocultos.
    - Modificar o realizar un diagnóstico a un caso determinado.
- Para Técnico:
    - Acceder a las métricas del sistema
    - Registrar usuarios con el rol “Especialista/Médico”

<u>El sistema no puede hacer:</u>

- Registrar usuarios con el rol “Médico”
- Registrar usuarios con el rol “Técnico”
- Permitir a los usuarios modificar su propia información

4. Alcance definitivo (IN / OUT)

La aplicación permitirá obtener un diagnóstico de una lesión cutánea a través de una imágen aportada por el usuario. Se podrá ver el rendimiento de la aplicación desde un panel de monitorización.

<u>Funcionalidades mínimas:</u>

- Los especialistas podrán registrar pacientes
- El administrador podrá registrar especialistas
- Realizar predicciones de lesiones cutáneas en base a imágenes
- Se guarda en la base de datos el historial de los pacientes
- Los técnicos pueden monitorizar las métricas de la aplicación
- Se recogen métricas de la aplicación

<u>Funcionalidades opcionales:</u>

- El sistema utiliza el feedback de los casos para ajustar el modelo de predicción
- Generar histórico de pacientes en base a casos, diagnósticos y pruebas
- Los pacientes pueden consultar su historial
- Los especialistas pueden consultar los historiales de todos los pacientes

<u>Elementos descartados:</u>

- La aplicación de administración del sistema se limitará a CRUD de usuarios
- Los usuarios que no sean administradores no podrán modificar datos de usuarios, tampoco de sus propios usuarios
- No se podrán descargar documentos asociados a los casos

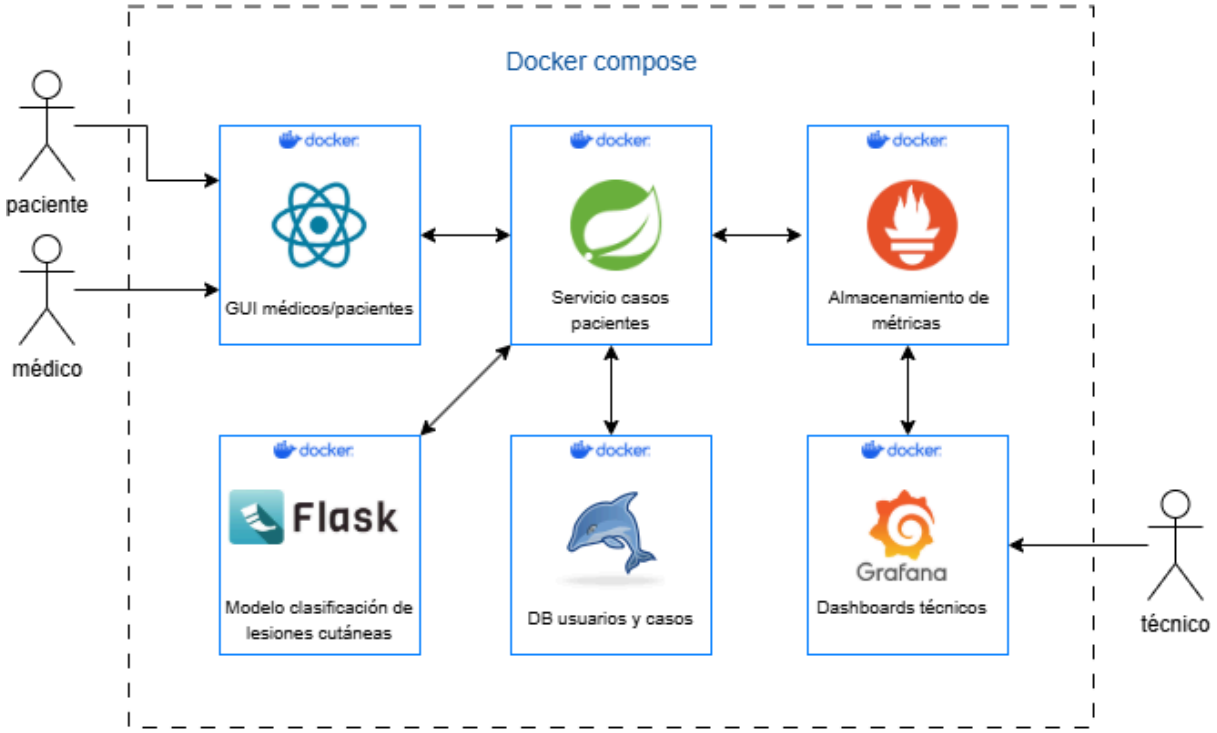
5. Arquitectura del sistema (alto nivel)

El sistema consta de varias aplicaciones, cada una de ellas en su propio contenedor de Docker, comunicándose entre sí dentro de la red interna que Docker establece y exponiendo al exterior los clientes que serán el punto de acceso de los usuarios: uno para médicos y pacientes y otro para que los técnicos puedan ver el rendimiento del sistema. De esta forma se consigue una mayor seguridad, ya que los usuarios no tienen forma de interaccionar directamente con la base de datos ni con los servicios.

- **GUI médicos/pacientes:** SPA desarrollada en React que permite a los pacientes consultar su historial de casos y a los médicos consultar el historial de casos de

todos los pacientes, así como crear nuevos diagnósticos en base a imágenes de lesiones y registrar nuevos pacientes.

- **Servicio casos pacientes:** aplicación de Spring que tiene una API REST para comunicarse con *GUI médicos/pacientes* así como con Modelo clasificación de lesiones cutáneas. Se comunica con la *DB usuarios y casos* para almacenar y recuperar información sobre los usuarios y sus casos. Expone métricas para que sean almacenadas por *Almacenamiento de métricas*.
- **Modelo clasificación de lesiones cutáneas:** aplicación de Flask que gestiona el modelo de ML que realiza la clasificación de imágenes de lesiones de la piel. Se comunica con *Servicio casos pacientes* a través de una API REST.
- **Almacenamiento de métricas:** una instancia de Prometheus que almacena las métricas expuestas desde *Servicio casos pacientes* para que puedan ser consultadas por *Dashboard técnicos*.
- **Dashboard técnicos:** una instancia de Grafana accesible por los técnicos para consultar los dashboards de las métricas del sistema. Su fuente de datos es la instancia de Prometheus *Almacenamiento de métricas.*



6. Tecnologías seleccionadas

Se han escogido las siguientes tecnologías teniendo en cuenta, en primer lugar, su aptitud para conseguir los objetivos del sistema y en segundo lugar la familiaridad de los componentes del equipo con ellas. La cantidad de tiempo limitada para el desarrollo del sistema hace que sea arriesgado optar por alternativas desconocidas para ver si presentan una mejora significativa en los resultados.

- **Lenguajes**
    - **Python:** para desarrollar el servicio que incluye el modelo de IA. Python es el lenguaje más empleado en el desarrollo de librerías dedicadas a la inteligencia artificial y el Big Data. Además de un extenso catálogo de librerías, existe una comunidad muy grande de usuarios que comparten su experiencia, facilitando el aprendizaje de éstas.
    - **Java:** para desarrollar la aplicación de backend que gestiona el grueso de la lógica de negocio. El framework Spring, desarrollado en este lenguaje, permite desarrollar una aplicación con funcionalidades a medida de forma rápida y segura.
    - **HTML + CSS:** diseño de las interfaz de pacientes y médicos. Uso imprescindible de estos lenguajes para el diseño web moderno.
    - **TS:** desarrollo del cliente web para pacientes y médicos. La aportación del tipado que TS impone sobre JS ayuda a la hora de desarrollar código libre de bugs y errores.
- **Frameworks**
    - **Spring:** aplicación principal del backend encargada del grueso de la lógica de negocio. A partir de la herramienta Spring Boot y sus starters, se puede levantar la base de la aplicación en un instante. Se podría haber utilizado también Python para el backend, empleando frameworks como Django, pero la existencia de Spring Boot facilita en gran medida desarrollar una aplicación típica que sirva de núcleo del sistema.
    - **Flask:** aplicación ligera encargada de gestionar el modelo de IA y exponerlo a la aplicación de backend principal. Mantener el modelo en esta aplicación desarrollada en Python, usando una API como interfaz con el resto del sistema nos permitirá aprovechar lo mejor de los dos mundos (Python y Java) sin acoplar los desarrollos. Únicamente habrá que definir bien las comunicaciones vía API entre ambas partes.
    - **React:** cliente web para médicos y pacientes. Permite el desarrollo de aplicaciones single page muy robustas y dinámicas. Dado el tipo de interacciones de los usuarios con nuestro sistema, un framework de SPA resultará en mejor rendimiento y experiencia de uso.
- **Librerías principales**
    - **PyTorch/Tensor Flow:** desarrollo del modelo de ML.
    - **Flask-RESTful:** API REST del servicio del modelo.
    - **Spring Security:** autenticación y autorización.
    - **Spring Web:** API REST de la aplicación principal de backend
    - **JUnit + Mockito:** test unitario en Spring
    - **SpringDoc OpenAPI:** permite documentar fácilmente la API, generando una página web de Swagger donde poder explorar ésta en detalle.
    - **Tailwind:** facilita la incorporación de estilos a los componentes propios del framework React. Agilizará el desarrollo.
- **Tipo de base de datos**
    - **MySQL:** base de datos relacional que nos permitirá guardar la información de los usuarios y sus casos de forma estructurada, tanto para realizar el seguimiento de los casos como para poder recuperar información para medir el rendimiento del modelo. Nos ha parecido el tipo de base de datos

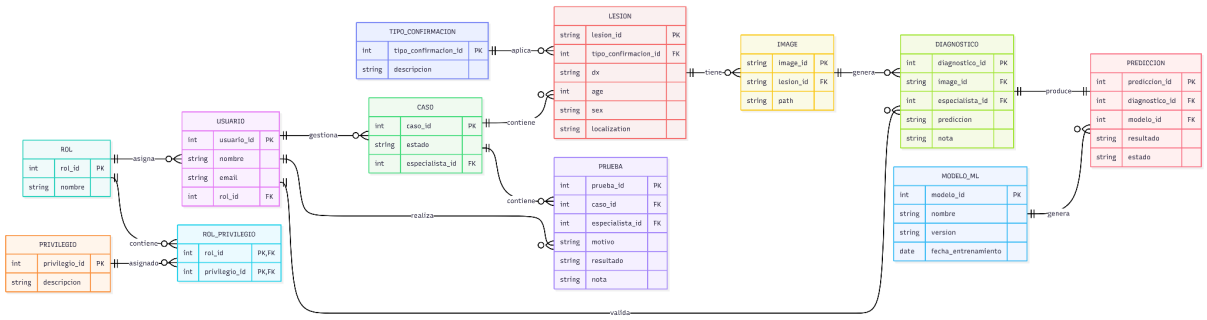
adecuado, ya que tienen mucho peso las entidades y, en segundo lugar, las relaciones entre estas.

- **Herramientas auxiliares**
  - **Grafana:** para desarrollar dashboards en los que visualizar las métricas de la aplicación almacenadas en Prometheus. Es muy buena opción en nuestro caso, ya que existe versión libre que no presenta limitaciones significativas.
  - **Prometheus:** recopila y almacena las métricas de la aplicación principal del backend. Similar a Grafana, es un software libre que nos aporta gran valor sin necesidad de incurrir en costes.
  - **Docker:** permitirá utilizar una arquitectura de contenedores comunicados dentro de una red interna. Se expondrán al exterior los clientes para los diferentes usuarios y se mantendrán el resto de componentes aislados del exterior.

7. Modelo de datos

Diagrama entidad-relación

Este diagrama compone las tablas resultantes tras el análisis realizado de cara a los requerimientos del proyecto, a partir de las necesidades del usuario a la vista del dashboard y la información relevante que ofrece el dataset.



Explicación de tablas

- **USUARIO:** Representa a los usuarios que intervienen en el sistema: especialistas, administradores, usuarios, etc.
  - **Campos principales:**
    - usuario_id (**PK**)
    - nombre
    - email
    - rol_id (**FK → ROL.rol_id**)
  - **Relaciones:**
    - Un usuario tiene un rol (**ROL**)
    - Un usuario (especialista) puede gestionar varios casos (**CASO**)
    - Un usuario puede realizar varias pruebas (**PRUEBA**)
    - Un usuario puede validar varios diagnósticos (**DIAGNOSTICO**)

- **ROL:** Define los roles de usuario dentro del sistema.
  - **Campos principales:**
    - rol_id (**PK**)
    - nombre
  - **Relaciones:**
    - Un rol puede asignarse a varios usuarios (**USUARIO**)
    - Un rol puede contener varios privilegios mediante **ROL_PRIVILEGIO**
- **PRIVILEGIO:** Representa las acciones o permisos que pueden asignarse a los roles.
  - **Campos principales:**
    - privilegio_id (**PK**)
    - descripcion
  - **Relaciones:**
    - Un privilegio puede estar asignado a varios roles (**ROL_PRIVILEGIO**)
- **ROL_PRIVILEGIO:** Tabla intermedia para la relación **many-to-many** entre **ROL** y **PRIVILEGIO**.
  - **Campos principales:**
    - rol_id (**PK, FK → ROL.rol_id**)
    - privilegio_id (**PK, FK → PRIVILEGIO.privilegio_id**)
  - **Relaciones:**
    - Une roles con privilegios asignados
- **CASO:** Representa un caso clínico gestionado por un especialista.
  - **Campos principales:**
    - `caso_id` (**PK**)
    - estado
    - especialista_id (**FK → USUARIO.usuario_id**)
  - **Relaciones:**
    - Un caso contiene varias pruebas (**PRUEBA**)
    - Un caso contiene varias lesiones (**LESION**)
    - Un caso es gestionado por un especialista (**USUARIO**)
- **PRUEBA:** Registra evaluaciones o pruebas realizadas sobre un caso por un especialista.
  - **Campos principales:**
    - prueba_id (**PK**)
    - caso_id (**FK → CASO.caso_id**)
    - especialista_id (**FK → USUARIO.usuario_id**)
    - motivo
    - resultado
    - nota
  - **Relaciones:**
    - Una prueba pertenece a un caso (**CASO**)
    - Una prueba es realizada por un especialista (**USUARIO**)
- **LESION:** Información de cada lesión dermatológica dentro de un caso.
  - **Campos principales:**
    - lesion_id (**PK**)
    - tipo_confirmacion_id (**FK → TIPO_CONFIRMACION.tipo_confirmacion_id**)
    - dx (diagnóstico final: akiec, bcc, bkl, etc.)
    - age

        - sex
        - localization
    - **Relaciones:**
        - Una lesión puede contener varias imágenes (**IMAGE**)
        - Una lesión pertenece a un caso (**CASO**)
        - Una lesión tiene un tipo de confirmación (**TIPO_CONFIRMACION**)
- **TIPO_CONFIRMACION:** Define el método de confirmación del diagnóstico de la lesión.
    - **Campos principales:**
        - tipo_confirmacion_id (**PK**)
        - descripcion (histo, follow_up, consensus, confocal)
    - **Relaciones:**
        - Un tipo de confirmación puede aplicarse a varias lesiones (**LESION**)
- **IMAGE:** Imágenes dermatoscopias asociadas a cada lesión.
    - **Campos principales:**
        - image_id (**PK**)
        - lesion_id (**FK → LESION.lesion_id**)
        - path (ruta del archivo)
    - **Relaciones:**
        - Una imagen pertenece a una lesión (**LESION**)
        - Una imagen puede generar varios diagnósticos (**DIAGNOSTICO**)
- **DIAGNOSTICO:** Registra diagnósticos realizados sobre una imagen por un especialista.
    - **Campos principales:**
        - diagnostico_id (PK)
        - image_id (FK → IMAGE.image_id)
        - especialista_id (FK → USUARIO.usuario_id)
        - prediccion
        - nota
    - **Relaciones:**
        - Un diagnóstico pertenece a una imagen (**IMAGE**)
        - Un diagnóstico es validado por un especialista (**USUARIO**)
        - Un diagnóstico puede producir varias predicciones (**PREDICCION**)
- **MODELO_ML:** Registra los modelos de machine learning entrenados para predicciones.
    - **Campos principales:**
        - modelo_id (**PK**)
        - nombre
        - version
        - fecha_entrenamiento
    - **Relaciones:**
        - Un modelo puede generar varias predicciones (**PREDICCION**)
- **REDUCCION:** Resultado de aplicar un modelo ML a un diagnóstico.
    - **Campos principales:**
        - prediccion_id (**PK**)
        - diagnostico_id (**FK → DIAGNOSTICO.diagnostico_id**)
        - modelo_id (**FK → MODELO_ML.modelo_id**)
        - resultado

        - estado
    - **Relaciones:**
        - Una predicción pertenece a un diagnóstico (**DIAGNOSTICO**)
        - Una predicción es generada por un modelo ML (**MODELO_ML**)

Diccionario de variables

- **image_id**: Identificador único de la imagen en el dataset ISIC/HAM10000. No debe haber duplicados en combinación con interaction_modality. (string)
- **lesion_id**: Identificador de la lesión asociada a la imagen. Permite vincular múltiples imágenes de la misma lesión. (string)
- **dx**: Diagnóstico final de la lesión. Valores: akiec, bcc, bkl, df, mel, nv, vasc. (categoría)
- **tipo_confirmacion**: Método usado para confirmar el diagnóstico: histo, follow_up, consensus, confocal. (categoría)
- **age**: Edad del paciente al momento de la imagen. (integer)
- **sex**: Sexo del paciente: male o female. (categoría)
- **localization**: Localización de la lesión en el cuerpo. (string)
- **path**: Ruta o nombre del archivo de la imagen en el dataset. (string)
- **prob_m_dx_<clase>**: Probabilidad predicha por la máquina (modelo CNN) para la clase específica (akiec, bcc, etc.). Incluye _mal para la suma de todas clases malignas. (float)
- **prob_h_dx_<clase>**: Probabilidad agregada de humanos para la clase específica, expresada en porcentaje. (float)
- **user_dx_without_interaction_<clase>**: Número de participantes humanos que seleccionaron este diagnóstico sin interacción con la IA. (integer)
- **user_dx_with_interaction_<clase>**: Número de participantes humanos que seleccionaron este diagnóstico con interacción con la IA. (integer)
- **interaction_modality**: Modalidad de interacción con el sistema IA: ResNet34 CNN (Malignancy Probability, Multi-Class Probability, CBIR), Human-Crowd Multi-Class. (string)
- **study_id**: Identificador del estudio o experimento (ej. ISIC2018_Task3, NatureMedicine 2020). (string)
- **segmentation_mask**: Máscara binaria de la lesión usada para segmentación y análisis de activaciones de CNN. (array/binary image)
- **modelo_id**: Identificador del modelo de machine learning usado para generar predicciones. (string)
- **prediccion_resultado**: Resultado de la predicción del modelo ML sobre un diagnóstico (maligno, benigno u otras clases específicas). (string)
- **prediccion_estado**: Estado de la predicción (pendiente, confirmada, rechazada). (string)

8. Datos

La fuente de datos que se emplearán es: *The HAM10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions*. Este dataset publicado por la Universidad de Harvard puede ser usado bajo licencia tipo *Creative Commons Attribution-NonCommercial 4.0 International Public License.* Contiene 10015

imágenes reales acompañadas del tipo de lesión del que se trata para poder entrenar modelos. El tipo de lesión ha sido determinado por especialistas mediante diferentes prácticas médicas.

Si con el dataset anterior tuviésemos problemas, podemos conseguir imágenes, también bajo licencias de tipo *Creative Commons,* de la página web del proyecto ISIC. Los participantes de este proyecto suben imágenes que consideran especialmente útiles para mejorar las herramientas dedicadas al diagnóstico del cáncer de piel, incluido el entrenamiento de especialistas, por lo que son de buena calidad para nuestro propósito.

Un aspecto muy útil de la colección de imágenes de ISIC es que permite filtrar qué tipo de imágenes se quieren obtener. Podemos sacar provecho de este filtro para reforzar puntos débiles que detectemos en el dataset HAM10000 como, por ejemplo, la falta de imágenes de algún tipo de lesión.

Las imágenes que se suban a la plataforma para realizar predicciones asociadas a los diagnósticos de los casos de los pacientes servirán para ajustar el modelo en base al feedback obtenido: predicción errónea o acertada.

9. Ética, legalidad y accesibilidad

Ética y legalidad

El desarrollo de un sistema de apoyo al diagnóstico dermatológico basado en inteligencia artificial requiere un marco sólido de responsabilidad ética, cumplimiento legal y accesibilidad universal. Este apartado recoge las principales consideraciones aplicadas en el proyecto.

- **Uso responsable de IA:**
  - El sistema está diseñado para asistir, no para reemplazar la labor del profesional médico.
  - El diagnóstico final siempre queda en manos de un especialista, evitando cualquier riesgo asociado al uso autónomo de herramientas de IA en contextos clínicos.
- **Minimización de sesgos y equidad:**
  - Se analiza el balance de clases y se aplican técnicas como *data augmentation* o recolección complementaria de datos.
  - Se evalúa el desempeño del modelo en diferentes subgrupos para identificar posibles disparidades.
  - Se incluye la supervisión del especialista como mecanismo clave de corrección.
- **Transparencia y explicabilidad:** El sistema mostrará los resultados del modelo junto con información relevante del proceso (probabilidad, clase predicha).
- **Seguridad del paciente:** El diseño busca minimizar daños derivados de errores de clasificación:
  - Revisión obligatoria por un médico.
  - Prioridad de la evaluación humana frente a la de la IA.
  - Monitorización continua del rendimiento del modelo para detectar degradación.

- **Protección de datos personales (RGPD/GDPR).** El sistema cumple con el Reglamento General de Protección de Datos (RGPD) y la legislación española vigente. Aplicando medidas como:
    - Consentimiento del paciente para el uso y almacenamiento de datos.
    - Encriptación de datos sensibles.
    - Acceso restringido según roles (médico, paciente, técnico).
- **Licencia de los datos:** Los datasets utilizados (HAM10000 e ISIC) están bajo licencias **Creative Commons** de tipo **Attribution-NonCommercial 4.0**, lo que permite su uso académico y no comercial, cumpliendo los requisitos del proyecto. El sistema respeta las condiciones de uso y no redistribuye el dataset original.

Accesibilidad

Para garantizar que el sistema pueda ser utilizado por el mayor número de usuarios posible, incluidos aquellos con limitaciones funcionales, se aplican medidas siguiendo la **WCAG 2.1 (Web Content Accessibility Guidelines)**:

- **Accesibilidad visual**
    - Contrastes adecuados entre texto y fondo.
    - Posibilidad de aumentar el tamaño de fuentes sin romper la interfaz.
    - Diseño claro y minimalista con enfoque en la legibilidad.
- **Accesibilidad motora**
    - Navegación completa mediante teclado.
    - Botones y elementos interactivos con área de clic ampliada.
- **Accesibilidad cognitiva**
    - Flujo simple y directo para subir imágenes y visualizar resultados.
    - Uso de iconografía clara y mensajes comprensibles.
    - Feedback contextual ante errores (p. ej., formato incorrecto de imagen).
- **Compatibilidad técnica**
    - Diseño responsive para uso desde dispositivos móviles, tablet o escritorio.
    - Estructura semántica adecuada en HTML que facilita el uso por lectores de pantalla.

10. Backlog inicial

Historias de usuario

1. Registro de pacientes
   Como médico
   quiero registrar nuevos pacientes en el sistema
   para poder crear diagnósticos y mantener su historial clínico digital.

   Criterios de aceptación:
   - El médico puede introducir datos básicos del paciente.
   - El paciente queda almacenado en la base de datos.

- El sistema permite acceder posteriormente a su historial.

2. <u>Consulta de historial clínico (paciente)</u>
   Como paciente
   quiero consultar mi historial de casos dermatológicos
   para revisar mis diagnósticos y su evolución.

   Criterios de aceptación:
   - El paciente puede ver todos sus casos registrados.
   - Cada caso incluye fecha, imagen y diagnóstico.
   - Solo el paciente autenticado puede ver su información.

3. <u>Consulta de historial clínico (médico)</u>
   Como médico
   quiero consultar el historial dermatológico de un paciente
   para analizar casos anteriores antes de realizar un nuevo diagnóstico.

   Criterios de aceptación:
   - El médico puede buscar pacientes registrados.
   - Puede acceder a todos los casos asociados.
   - Puede visualizar imágenes y diagnósticos previos.

4. <u>Creación de un nuevo caso clínico</u>
   Como médico
   quiero registrar un nuevo caso dermatológico para un paciente
   para documentar una nueva lesión o consulta.

   Criterios de aceptación:
   - El médico selecciona un paciente.
   - Puede crear un diagnóstico para éste.
   - El caso queda registrado en el sistema.

5. <u>Análisis automático de imágenes</u>
   Como médico
   quiero que el sistema analice automáticamente la imagen de una lesión
   para obtener una clasificación preliminar que apoye el diagnóstico.

   Criterios de aceptación:
   - La imagen se envía al modelo de ML.
   - El sistema devuelve una clasificación de la lesión.
   - El resultado se almacena junto al diagnóstico.

6. <u>Visualización del resultado del modelo</u>
   Como médico
   quiero ver el resultado de la clasificación generada por la IA
   para utilizarla como apoyo en mi diagnóstico final.

   Criterios de aceptación:

- El resultado se muestra junto a la imagen analizada.
- Se muestra el tipo de lesión identificado.

7. Actualización del estado del diagnóstico
   Como médico
   quiero marcar el estado del diagnóstico
   para dejar constancia del resultado clínico.

   Criterios de aceptación:
   - El médico puede introducir su evaluación.
   - El estado del diagnóstico queda guardado en el historial del paciente.
   - El caso se marca como completado.

8. Consulta de resultados por el paciente
   Como paciente
   quiero ver los diagnósticos de mis casos registrados
   para conocer los resultados de mis consultas dermatológicas.

   Criterios de aceptación:
   - El paciente puede ver la información del diagnóstico.
   - Se muestra la fecha y el médico responsable.
   - La información es de solo lectura.

9. Comunicación entre servicios del sistema
   Como sistema
   quiero que el servicio de casos se comunique con el modelo de clasificación
   para procesar imágenes y obtener resultados automáticamente.

   Criterios de aceptación:
   - La comunicación se realiza mediante API REST.
   - El servicio gestiona errores de comunicación.
   - Se devuelve una respuesta estructurada con la clasificación.

10. Exposición de métricas del sistema
    Como sistema
    quiero exponer métricas del servicio de casos
    para que puedan ser monitorizadas por el sistema de observabilidad.

    Criterios de aceptación:
    - El servicio expone métricas compatibles con Prometheus.
    - Se registran métricas de uso y rendimiento.
    - Las métricas pueden ser recolectadas automáticamente.

11. Monitorización del sistema
    Como técnico del sistema
    quiero visualizar métricas del sistema en dashboards
    para monitorizar el rendimiento y detectar problemas.

Criterios de aceptación:

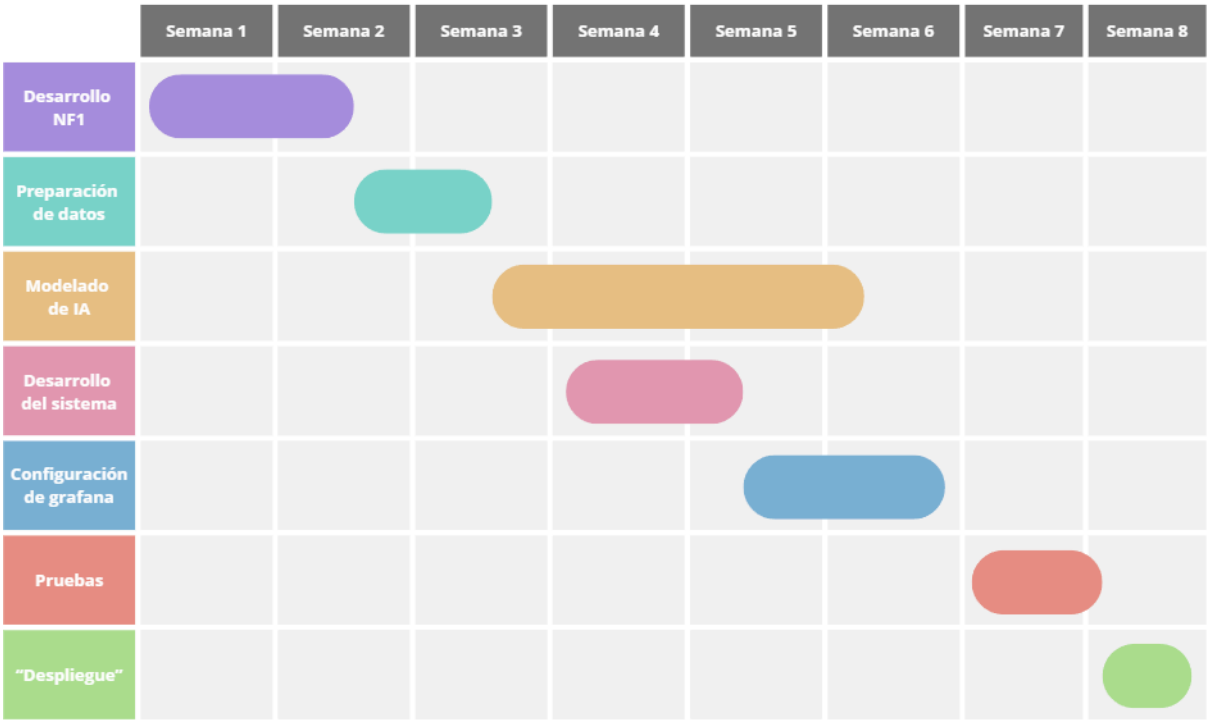
- Los dashboards muestran métricas de uso.
- Se muestran tiempos de respuesta del servicio.
- Los datos provienen de Prometheus.

12. <u>Análisis del rendimiento del modelo</u>
    Como técnico del sistema
    quiero consultar métricas del uso del modelo de clasificación
    para evaluar su rendimiento y detectar posibles fallos.

    Criterios de aceptación:

    - Se muestran métricas de número de clasificaciones realizadas.
    - Se muestran tiempos de respuesta.
    - Las métricas pueden visualizarse en Grafana.

11. Plan de trabajo



1. Desarrollo de la Fase 1
2. Limpieza y preparación de los datos para el modelo
3. Desarrollo y entrenamiento del modelo IA
4. Desarrollo del frontend y el sistema que se comunicará con el modelo
5. Configuración de métricas para grafana
6. Últimas pruebas para comprobación final
7. Para este punto la aplicación completa estaría terminada

Es posible que durante ciertas etapas haya inconvenientes que nos hagan retrasar otras tareas. Lo bueno es que tanto el modelado de IA como el desarrollo del sistema son tareas

que se pueden ir avanzando de manera independiente, si por algún problema el modelado o desarrollo del sistema se retrasa un poco se puede ir avanzando con la otra tarea.

12. Estrategia de desarrollo

- **Flujo de Trabajo:**
  - Main: Contiene el código probado, se actualiza con cada hito.
  - Desarrollo: Rama donde se fusionan las nuevas funcionalidades antes de pasar a producción.
- **Estrategia de integración:**
  - Cada solicitud de PR deberá incluir una descripción con los cambios realizados.
  - Para un merge en main se puede revisar entre varios miembros del equipo.
  - Integraciones por cada hito superado.
- **Validación de funcionalidad:**
  - Establecer criterios de aceptación.
  - Establecer revisiones completas de funcionalidad cada 1-2 semanas.
- **Testing básico:**
  - Pruebas unitarias (Backend/Modelo IA)
    - Verificar las funciones de preprocesamiento de la imagen.
    - Verificar que el LogIn/Registro mantenga el token con la información y permisos del usuario.
  - Pruebas de integración
    - Comprobar que la imágen subida en el front llegue al modelo.
    - Comprobar que el modelo manda el diagnóstico correctamente.

13. Riesgos técnicos actualizados

- **Riesgo de sesgos por desbalanceo de clases:**
  - El dataset contiene muchos registros de una enfermedad concreta.
  - Impacto muy alto y alta probabilidad
  - El modelo podría pasar por alto ciertas enfermedades menos frecuentes.
- **Baja calidad o variabilidad en las fotografías:**
  - Fotos borrosas, con diferente/baja iluminación
  - Alto impacto y probabilidad media.
  - Reduce considerablemente la fiabilidad de la predicción.
  - Aplicar filtros de preprocesamiento de imágenes, para normalizar las características de las imágenes.
- **Degradación del modelo:**
  - El modelo pierde precisión con cámaras diferentes o colores de piel no vistos antes.
  - Alto impacto y baja probabilidad.
  - El modelo se vuelve obsoleto o peligroso con el tiempo.
  - Establecer un sistema de monitoreo para reentrenar el modelo con el tiempo
- **Latencia en el procesamiento de imágenes:**
  - Si el tiempo de espera es muy alto se verán retrasadas las acciones posteriores al procesamiento de las imágenes.
  - Riesgo y probabilidad medios.

- Afecta a la experiencia del usuario y a la eficiencia clínica.
- Optimizar el modelo para una respuesta más rápida, sin descompensar el % de confianza en el diagnóstico.

14. Repositorio del proyecto

https://github.com/Gyara2/skin-disease-detection-ml/blob/main/README.md#frameworks